

Scalability & Multi-Tenancy Architecture Review

School App — readiness for multiple schools and lakhs of students

Flutter + Firebase (Firestore / Auth / Cloud Functions / FCM)

Firebase project: **attendanceapp-e76e1**

Review date: 11 June 2026 · Branch: main

Verdict: The foundation is multi-tenant and well-secured, but the app will **not** handle a large school as-is. Two BLOCKER-level data-loading limits cap every school at 500 students and make dashboards read the entire school on every open. Both are fixable without re-architecting. Details inside.

1 Executive summary

Your app is already built as a proper multi-tenant system: every business collection lives under `schools/{schoolId}/...`, the Firestore security rules enforce strict per-tenant isolation ("Phase 3"), and expensive operations (cascade delete, promotion, push) are batched correctly. That is the hard part, and it is done well.

The problem is **read scale**. The client was written to load a whole school's students into memory and compute every total in Dart. That works for a 200-student demo school; it breaks — silently and then expensively — as you grow toward thousands of students per school and many schools per project.

There are **12 findings** below. The two BLOCKERs must be fixed before you onboard any school above ~500 students. The HIGH items are needed before you market to large schools or cross ~5,000 total students. MEDIUM/LOW are hardening for the lakhs-of-students horizon.

Severity counts

BLOCKER	HIGH	MEDIUM	LOW
2	3	5	2

Findings at a glance

ID	Finding	Severity	Effort
SCALE-01	500-student hard cap silently truncates every school	BLOCKER	S
SCALE-02	Dashboards read the entire school client-side on open	BLOCKER	M
SCALE-03	Whole-school real-time listener on dashboards	HIGH	M
SCALE-04	No server-side aggregation / rollup counters	HIGH	L
SCALE-05	Rules re-read <code>allowed_users</code> instead of using claims	HIGH	M
SCALE-06	Client 'school_1' fallbacks are wrong-tenant landmines	MEDIUM	S
SCALE-07	No scheduled data retention / archival	MEDIUM	M
SCALE-08	Firestore region <code>us-central1</code> (latency from India)	MEDIUM	L*
SCALE-09	Year-rollover promotion is client-driven, sequential	MEDIUM	M
SCALE-10	<code>fetchByRolls</code> uses N+1 single-doc reads	LOW	S
SCALE-11	<code>allowed_users</code> is one global hot collection	MEDIUM	M
SCALE-12	Today-summary fan-out scales with section count	LOW	S

Effort: S = hours · M = a few days · L = under a day · L* = decision now, cheap if done before launch.

2 What is already done well (do not regress these)

So a future change does not undo working design, here is what the audit confirms is solid:

Area	Why it scales
Tenant data layout	Every business collection is nested under <code>schools/{sid}/...</code> via <code>BaseFirestoreService.schoolCollection()</code> . Schools are fully separated at the data level.
Strict rule isolation	Firestore rules fail <i>closed</i> : a missing <code>schoolId</code> denies access rather than defaulting to <code>school_1</code> . Only the hardcoded root admin crosses schools (<code>firestore.rules, inSchool()</code>).
Attendance modelled per day	Attendance is one document per class+section+date (<code><class>_<section>_<Y-M-D></code>). This avoids the classic 1 MB unbounded-document trap of one-doc-per-class-forever.
Push via FCM topics	Notifications publish to a topic <code>s_{sid}_{audience}</code> (<code>functions/index.js</code>). One message regardless of device count — no per-device fan-out.
Batched cascades	Student delete, promotion and purge all chunk writes to stay under Firestore's 500-op batch limit (chunks of 200/400).
Short-TTL read cache	Past attendance day-docs are cached 30 s and namespaced by school, collapsing repeated dashboard reads (<code>student_service.dart</code>).
Custom claims pipeline	<code>syncUserClaims</code> already stamps role + <code>schoolId</code> onto the Auth token on every <code>allowed_users</code> write — the groundwork for cheaper rules is in place (see SCALE-05).

3 Findings — detail

Each finding is self-contained. Hand any subset back to me by ID (e.g. "do SCALE-01 and SCALE-02") and I will have everything I need to implement it — the location, intent and acceptance test are all here.

SCALE-01

500-student hard cap truncates every school

BLOCKER ·
Effort: S

Where

```
lib/repositories/student_repository.dart:226 → fetchAll(): _students.limit(500).get()  
lib/repositories/student_repository.dart:432 → watchAll(): _students.limit(500).snapshots()
```

What happens at scale

Every code path that lists "all students" stops at 500. A school with 600, 5,000 or 200,000 students will only ever load the first 500 (ordered arbitrarily). Rosters, the coordinator/principal/owner dashboards, analytics and today's attendance summary all silently drop the rest. This is data loss, not just slowness — and it fails quietly, which is worse: nobody sees an error, the numbers are just wrong.

Why it happens

The cap was a stop-gap to bound an unscoped whole-collection read. It is the right instinct (never read an unbounded collection) but the wrong mechanism — a fixed truncation instead of pagination or aggregation.

Fix

Stop loading whole-school student lists at all for dashboards (see SCALE-02/04). Where a full list is genuinely needed, switch to **cursor pagination** — the repository already has `fetchByClassPaginated()`; add the same `startAfter` loop to `fetchAll`, or page per class. For real-time, never `watchAll()` an unbounded collection; watch per class/section only.

Done when

A school with 1,000+ students shows correct totals and a complete roster; no query in the app uses a bare `.limit(500)` on students.

SCALE-02**Dashboards read the entire school on every open****BLOCKER** · Effort: M**Where**

```
lib/services/student_service.dart:1196 loadTodayFullSummary() → _repo.fetchAll() callers:
coordinator_dashboard.dart:167 · principal_dashboard.dart:195 · owner_home.dart:439 ·
owner_principal_home.dart:267 · analytics_screen.dart:115
```

What happens at scale

Opening a dashboard fetches *every student document in the school*, groups them in Dart, then fetches one attendance doc per class-section. For a 5,000-student school that is 5,000+ document reads **per dashboard open, per user, per refresh**. At lakhs of students this is both impossibly slow on the client and a serious Firestore bill (reads are billed per document). Several roles open these screens many times a day.

Why it happens

Totals (present / absent / leave / class strength) are computed by reading raw student and attendance documents into the client and counting them, rather than reading a small pre-aggregated summary.

Fix

Maintain **rollup counter documents** updated by Cloud Functions as attendance is marked and students are added/removed — e.g. `schools/{sid}/daily_summaries/{date}` holding per-class present/absent/total. Dashboards then read a handful of summary docs instead of the whole school. For class strength, also see SCALE-04 (`count()` aggregate queries) as a lighter interim step.

Done when

Opening any dashboard costs a fixed, small number of reads (tens, not thousands) regardless of school size; verified with a 5,000-student seed.

SCALE-03**Whole-school real-time listener on dashboards****HIGH** · Effort: M**Where**

```
lib/screens/coordinator_dashboard.dart:88 StudentService.instance.watchStudents().listen(...)
lib/screens/principal_dashboard.dart:85 StudentService.instance.watchStudents().listen(...)
```

What happens at scale

These screens hold an open real-time listener over the whole-school students collection (currently capped at 500 by SCALE-01). A snapshot listener re-delivers and re-bills documents as the collection changes; over a large, busy school it streams far more data than a dashboard needs and keeps a costly live query open the whole time the screen is mounted.

Why it happens

The dashboards want a live student *count* / live class strength, but subscribe to the full document stream to get it.

Fix

Replace the whole-school stream with either (a) a listener on the small rollup/summary doc from SCALE-02, or (b) a periodic `count()` aggregate. Keep real-time document streams scoped to a single class/section (`watchStudentsByClass`), which is naturally bounded.

Done when

No screen subscribes to `watchStudents()/watchAll()`; dashboards update from a bounded summary source.

SCALE-04**No server-side aggregation anywhere****HIGH · Effort: L****Where**

```
grep '.count()' | AggregateQuery' across lib/ → 0 matches
```

What happens at scale

Firestore has supported server-side `count()` aggregation for years, but the app computes every count by downloading documents and counting them in Dart. This is the root cause shared by SCALE-01/02/03: counts scale with document volume instead of being $O(1)$.

Why it happens

The codebase predates (or didn't adopt) aggregate queries, so 'how many students / how many absent' is always answered by reading the underlying documents.

Fix

Adopt `Query.count()` for cheap totals (class strength, students per section, pending requests) as the quick win, and rollup counters (SCALE-02) for hot, frequently-read aggregates like daily attendance. As a rule: a screen that only needs a *number* must never download the *rows*.

Done when

Class-strength and 'how many' figures come from `count()` or a counter doc, with zero per-row reads.

SCALE-05**Security rules re-read `allowed_users` instead of using claims****HIGH · Effort: M****Where**

```
firebase.rules getUserData() → get(/allowed_users/{email}) (called by every rule)
functions/index.js:31 syncUserClaims → already sets {role, schoolId} custom claims
```

What happens at scale

Every rule evaluation calls `getUserData()`, which does a document `get()` of `allowed_users`. Rules are also bumping against Firestore's 1000-expression-per-request ceiling — the file is full of comments about teacher writes being denied because identity helpers re-derive `getUserData()` too many times. At lakhs of attendance writes a day this adds latency, fragility and a billed read to every protected operation.

Why it happens

Identity (role, schoolId) is resolved by reading a Firestore document inside the rule, even though the same values are already minted onto the Auth token by `syncUserClaims` and ignored by the rules.

Fix

Read identity from `request.auth.token.role / .schoolId` (custom claims) instead of `get()`. This removes a document read and most of the expression-count pressure from every rule. Keep the `get()` only as a fallback for accounts whose claims haven't propagated yet, and force a token refresh after provisioning.

Done when

Hot-path rules (attendance write, student read) evaluate with no `get()`; rule unit tests still pass; the 1000-expression comments become obsolete.

SCALE-06**Client 'school_1' fallbacks are wrong-tenant landmines****MEDIUM · Effort: S****Where**

```
auth_service.dart:121 return 'school_1' · student_service.dart:582 ?? 'school_1'
push_service.dart:73 ?? 'school_1' · owner_principal_home.dart:289/659/822/846 ?? 'school_1'
admin_login_screen.dart:76 currentSchoolId = 'school_1'
```

What happens at scale

If currentSchoolId is ever null at the moment a query runs (session restore race, logout/login overlap), the client silently targets the literal tenant school_1. With the new fail-closed rules this no longer leaks data, but it produces confusing silent permission-denials and, in the worst timing, writes aimed at the wrong school. As you add real schools, school_1 becomes just another tenant and these defaults become genuinely dangerous.

Why it happens

A historical single-tenant default ('school_1') was left as a fallback in client code paths that should now require an explicit, resolved schoolId.

Fix

Replace every ?? 'school_1' / hardcoded assignment with a hard failure (throw / guard that blocks the action and surfaces 'no active school') when schoolId is null. Fail loud, never default. The notifier in BaseFirestoreService already lets screens wait for a real value.

Done when

No occurrence of the string 'school_1' as a fallback remains in lib/; a null schoolId blocks the operation with a clear message.

SCALE-07**No scheduled data retention / archival****MEDIUM · Effort: M****Where**

```
functions/index.js:650 purgeOldData → onCall (manual only, not scheduled)
```

What happens at scale

Attendance, notifications, audit_logs and deleted-student tombstones grow forever, per school. A purge function exists but must be invoked by hand. At lakhs of students × daily writes, these collections balloon, slowing range queries and inflating storage. Old data is rarely read but never aged out.

Why it happens

Retention was implemented as an on-demand admin action rather than an automatic policy.

Fix

Convert retention to a onSchedule (Cloud Scheduler) function that runs nightly per school, archiving or deleting beyond a retention window (e.g. keep raw attendance 18 months, summaries forever). Pair with the rollup docs from SCALE-02 so history survives as cheap aggregates after raw docs are aged out.

Done when

A scheduled job ages out data automatically; collection sizes stay bounded by the retention window in a load test.

SCALE-08**Firestore region us-central1 adds latency from India****MEDIUM · Effort: L* (decision now)****Where**

```
functions/index.js region: 'us-central1' (all triggers) · Firestore DB location fixed at project creation
```

What happens at scale

Because the client does many sequential round-trips (read students, then per-class attendance, then counts), latency multiplies. A US region adds ~200–300 ms per round trip for Indian users; ten chained reads becomes seconds of perceived lag. Firestore's location is chosen **once, at project creation, and cannot be changed** — so this is a now-or-never call.

Why it happens

The Firebase project defaulted to a US multi-region; for an India-focused school business the data lives far from its users.

Fix

If you are early enough to recreate the project, provision Firestore in asia-south1 (Mumbai) or asia-south2 (Delhi) and deploy functions to the matching region. If the project already holds real data, treat this as a documented trade-off and lean harder on SCALE-02/04 to cut round-trip count instead. Decide deliberately before launch.

Done when

A conscious, recorded decision: either Firestore is in an India region, or the latency is accepted and mitigated by fewer round-trips.

SCALE-09**Year-rollover promotion is client-driven and sequential****MEDIUM · Effort: M****Where**

```
lib/repositories/student_repository.dart promoteStudentsAtomic() → chunks of 200, awaited one batch at a time on the client
```

What happens at scale

Promotion is correctly chunked (200 students/batch, 2 ops each), but it runs on the client, one batch after another. Promoting a whole large school (thousands of students) means dozens-to-hundreds of sequential committed batches while the app stays open — slow, interruptible (backgrounding, network drop), and it cannot atomically span the whole school.

Why it happens

Annual promotion is a bulk back-office operation implemented as a foreground client loop.

Fix

Move whole-school promotion to a Cloud Function (callable or scheduled) using the Admin SDK and a BulkWriter, so it runs server-side reliably and resumably, independent of the client staying open. Keep the client path for small single-class promotions.

Done when

Promoting a 5,000-student school completes server-side without the app open and is resumable on failure.

SCALE-10**fetchByRolls uses N+1 single-document reads****LOW · Effort: S****Where**

```
lib/repositories/student_repository.dart:264 fetchByRolls →
Future.wait(rolls.map(fetchByRoll))
```

What happens at scale

Each roll is fetched with its own `get()`. Fine for one class (~40 students), but it is an $O(n)$ read pattern that will be reused in larger contexts and bills one read per roll.

Why it happens

Convenience implementation that loops single-document reads instead of a batched query.

Fix

Use `whereIn` on the roll/doc-id (batched in groups of 30, Firestore's `whereIn` limit) to fetch a class's rolls in 1–2 queries. Low priority, but a trivial efficiency win when touched.

Done when

Fetching N rolls costs $\text{ceil}(N/30)$ queries, not N reads.

SCALE-11**allowed_users is one global hot collection****MEDIUM · Effort: M****Where**

Root collection `allowed_users/{email}` – not school-scoped; read by every auth resolution and (today) every security-rule evaluation

What happens at scale

All users of all tenants share one flat root collection, and it is read on every login and (until SCALE-05) every rule check. Firestore handles large flat collections fine for key-lookups, so this is not urgent — but it is a cross-tenant hot path and a single blast radius if any code ever lists it unscoped.

Why it happens

Identity was intentionally kept global (email is globally unique, and rules resolve identity by email) — a reasonable choice that just needs guarding as volume grows.

Fix

Primarily mitigated by SCALE-05 (claims remove the per-rule read). Additionally: never run an unbounded list query on `allowed_users`; always look up by email key. Optionally store a `schoolId` field (already present) and index it for the rare admin 'list users in my school' view, scoped per tenant.

Done when

No unscoped list of `allowed_users` exists; rule evaluations no longer read it on the hot path.

SCALE-12**Today-summary fan-out scales with section count****LOW · Effort: S****Where**

```
lib/services/student_service.dart:1233 Future.wait(allKeys.map((k) =>
_attendance.doc(k).get()))
```

What happens at scale

loadTodayFullSummary issues one attendance-doc read per class-section for today. A large school with many sections (e.g. 60) does 60 reads just for today's summary, on top of the whole-student fetch from SCALE-02. Bounded by section count, so it is LOW on its own — but it compounds SCALE-02.

Why it happens

Today's totals are assembled by reading each section's live attendance doc directly.

Fix

Subsumed by the SCALE-02 rollup: a single daily_summaries/{date} doc per school removes this fan-out entirely. Until then, it is acceptable.

Done when

Today's summary reads one (or a few) rollup docs, not one-per-section.

4 Suggested order of work

You do not have to do all of this at once. Each phase unlocks the next tier of school size. Hand me a phase (or individual IDs) and I will implement it.

Phase A — Unblock growth past 500 students

SCALE-01, SCALE-02,
SCALE-04

Kills the silent 500-cap and stops dashboards reading whole schools. Adopt count() aggregates and a daily rollup doc. After this you can safely onboard schools of a few thousand students. This is the minimum to call the app 'scalable'.

Phase B — Cut cost & latency at the read layer

SCALE-03, SCALE-05,
SCALE-12

Remove the whole-school live listener, move rule identity onto custom claims (the pipeline already exists), and fold the today fan-out into the rollup. Big drop in Firestore reads/bill and rule fragility.

Phase C — Operational hardening for the long haul

SCALE-06, SCALE-07,
SCALE-09, SCALE-11

Remove school_1 fallbacks, schedule retention, move year-rollover to a server job, and guard the global identity collection. These keep a multi-school system healthy over years of data growth.

Phase D — Strategic / one-time decisions

SCALE-08, SCALE-10

Decide the Firestore region deliberately (now-or-never) and tidy the N+1 read when convenient. Low urgency, but SCALE-08 must be a conscious choice before real data lands.

5 How to hand this back to me

This report is built so you can return it to me and I will know exactly what to do. Any of these work:

- **By ID** — "Implement SCALE-01 and SCALE-02." Each finding above carries its location, intent, fix and a 'Done when' acceptance test, so I have full context.
- **By phase** — "Do Phase A." I will work the listed IDs in order and verify each against its acceptance criterion.
- **By goal** — "Make it handle 10,000 students per school." I will map that to Phases A–B and confirm scope before starting.

Before implementing, I will restate the finding, propose the concrete code change (files + approach), and — for anything that touches rules, functions or data shape — outline how to verify it with a seeded large-school test before you ship.

Bottom line: the architecture is sound and genuinely multi-tenant — you are not facing a rewrite. The blockers are a 500-row truncation and a read-the-whole-school dashboard pattern, both fixable with pagination, aggregate queries and rollup counters. Fix Phase A and you can confidently sell to multi-thousand-student schools; complete B–D and the lakhs-of-students, many-schools future is well within reach.